



This is an excerpt from the Python Data Science Handbook (<http://shop.oreilly.com/product/0636920034919.do>) by Jake VanderPlas; Jupyter notebooks are available on GitHub (<https://github.com/jakevdp/PythonDataScienceHandbook>).

The text is released under the CC-BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/3.0/us/legalcode>), and code is released under the MIT license (<https://opensource.org/licenses/MIT>). If you find this content useful, please consider supporting the work by buying the book (<http://shop.oreilly.com/product/0636920034919.do>)!

What Is Machine Learning?

< [Machine Learning \(05.00-machine-learning.html\)](#) | [Contents \(index.html\)](#) | [Introducing Scikit-Learn \(05.02-introducing-scikit-learn.html\)](#) >

 [Open in Colab](#)

(<https://colab.research.google.com/github/jakevdp/PythonDataScienceHandbook/blob/master/notebooks/What-Is-Machine-Learning.ipynb>)

Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain.

Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories of Machine Learning

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into *classification* tasks and *regression* tasks: in classification,

the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as *clustering* and *dimensionality reduction*. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

In addition, there are so-called *semi-supervised learning* methods, which falls somewhere between supervised learning and unsupervised learning. Semi-supervised learning methods are often useful when only incomplete labels are available.

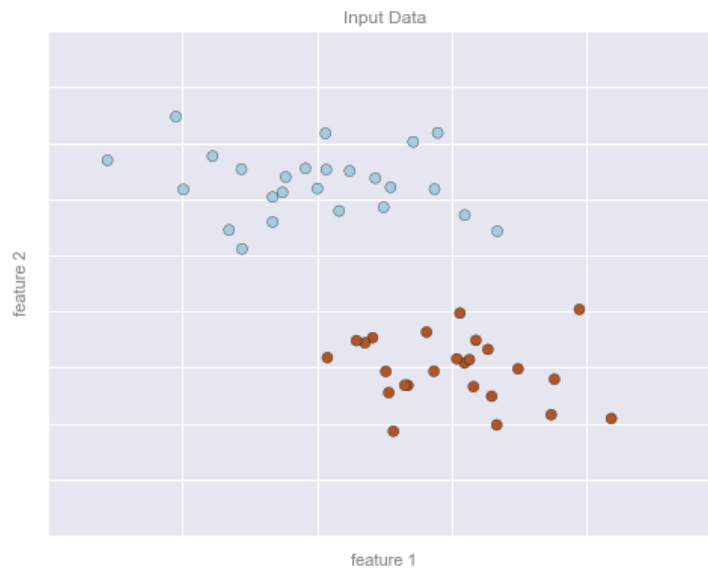
Qualitative Examples of Machine Learning Applications

To make these ideas more concrete, let's take a look at a few very simple examples of a machine learning task. These examples are meant to give an intuitive, non-quantitative overview of the types of machine learning tasks we will be looking at in this chapter. In later sections, we will go into more depth regarding the particular models and how they are used. For a preview of these more technical aspects, you can find the Python source that generates the following figures in the [Appendix: Figure Code \(06.00-figure-code.html\)](#).

Classification: Predicting discrete labels

We will first take a look at a simple *classification* task, in which you are given a set of labeled points and want to use these to classify some unlabeled points.

Imagine that we have the data shown in this figure:

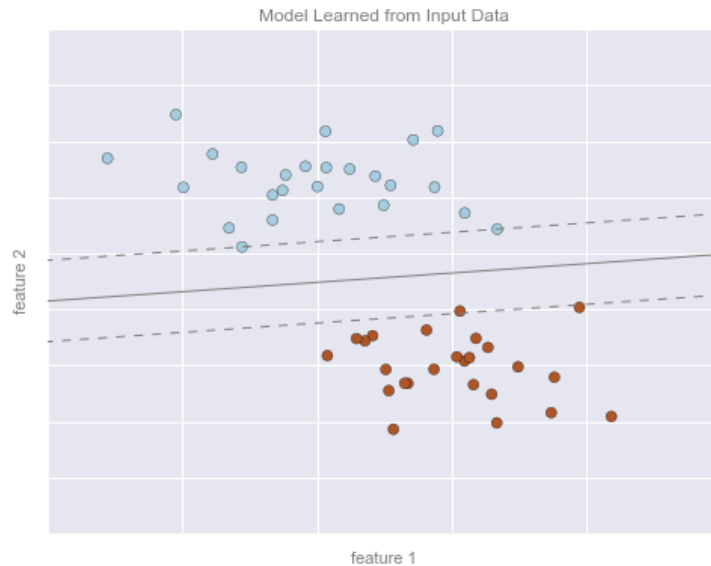


[figure source in Appendix \(06.00-figure-code.html#Classification-Example-Figure-1\)](#).

Here we have two-dimensional data: that is, we have two *features* for each point, represented by the (x,y) positions of the points on the plane. In addition, we have one of two *class labels* for each point, here represented by the colors of the points. From these features and labels, we would like to create a model that will let us decide whether a new point should be labeled "blue" or "red."

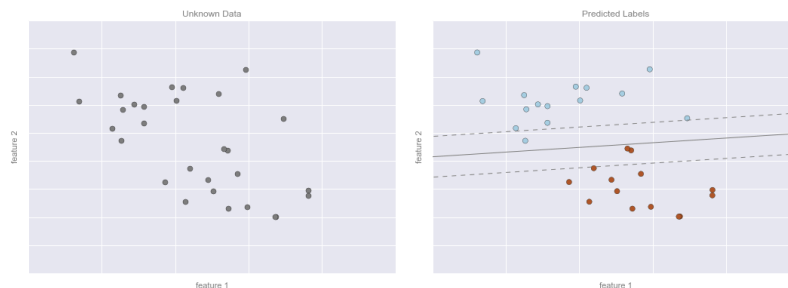
There are a number of possible models for such a classification task, but here we will use an extremely simple one. We will make the assumption that the two groups can be separated by drawing a straight line through the plane between them, such that points on each side of the line fall in the same group. Here the *model* is a quantitative version of the statement "a straight line separates the classes", while the *model parameters* are the particular numbers describing the location and orientation of that line for our data. The optimal values for these model parameters are learned from the data (this is the "learning" in machine learning), which is often called *training the model*.

The following figure shows a visual representation of what the trained model looks like for this data:



[figure source in Appendix \(06.00-figure-code.html#Classification-Example-Figure-2\).](#)

Now that this model has been trained, it can be generalized to new, unlabeled data. In other words, we can take a new set of data, draw this model line through it, and assign labels to the new points based on this model. This stage is usually called *prediction*. See the following figure:



[figure source in Appendix \(06.00-figure-code.html#Classification-Example-Figure-3\).](#)

This is the basic idea of a classification task in machine learning, where "classification" indicates that the data has discrete class labels. At first glance this may look fairly trivial: it would be relatively easy to simply look at this data and draw such a discriminatory line to accomplish this classification. A benefit of the machine learning approach, however, is that it can generalize to much larger datasets in many more dimensions.

For example, this is similar to the task of automated spam detection for email; in this case, we might use the following features and labels:

- *feature 1*, *feature 2*, etc. $\rightarrow$ normalized counts of important words or phrases ("Viagra", "Nigerian prince", etc.)
- *label* $\rightarrow$ "spam" or "not spam"

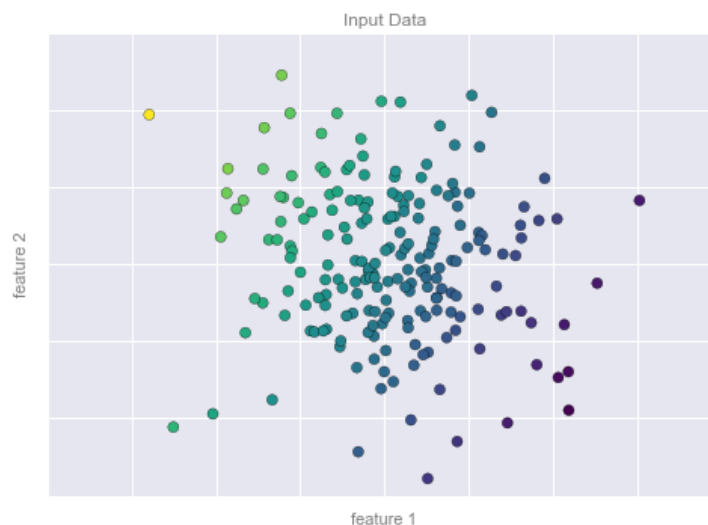
For the training set, these labels might be determined by individual inspection of a small representative sample of emails; for the remaining emails, the label would be determined using the model. For a suitably trained classification algorithm with enough well-constructed features (typically thousands or millions of words or phrases), this type of approach can be very effective. We will see an example of such text-based classification in [In Depth: Naive Bayes Classification \(05.05-naive-bayes.html\)](#).

Some important classification algorithms that we will discuss in more detail are Gaussian naive Bayes (see [In Depth: Naive Bayes Classification \(05.05-naive-bayes.html\)](#)), support vector machines (see [In-Depth: Support Vector Machines \(05.07-support-vector-machines.html\)](#)), and random forest classification (see [In-Depth: Decision Trees and Random Forests \(05.08-random-forests.html\)](#)).

Regression: Predicting continuous labels

In contrast with the discrete labels of a classification algorithm, we will next look at a simple *regression* task in which the labels are continuous quantities.

Consider the data shown in the following figure, which consists of a set of points each with a continuous label:

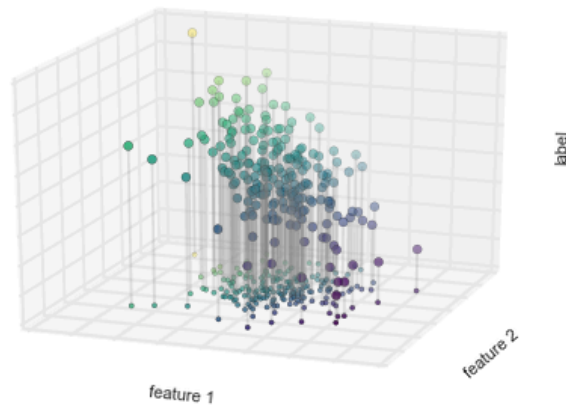


[figure source in Appendix \(06.00-figure-code.html#Regression-Example-Figure-1\)](#)

As with the classification example, we have two-dimensional data: that is, there are two features describing each data point. The color of each point represents the continuous label for that point.

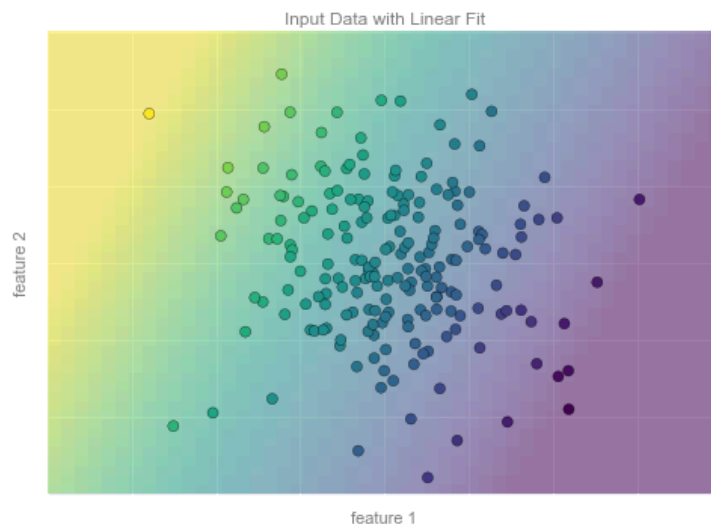
There are a number of possible regression models we might use for this type of data, but here we will use a simple linear regression to predict the points. This simple linear regression model assumes that if we treat the label as a third spatial dimension, we can fit a plane to the data. This is a higher-level generalization of the well-known problem of fitting a line to data with two coordinates.

We can visualize this setup as shown in the following figure:



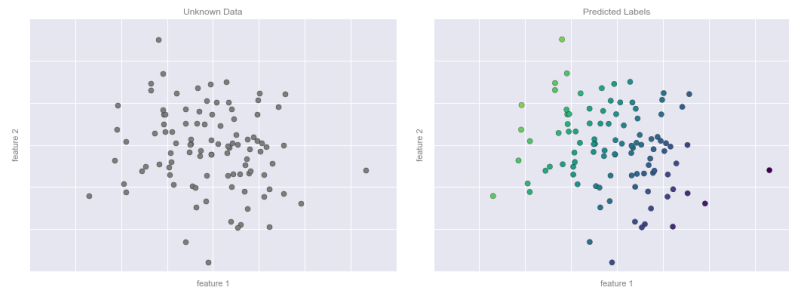
[figure source in Appendix \(06.00-figure-code.html#Regression-Example-Figure-2\).](#)

Notice that the *feature 1-feature 2* plane here is the same as in the two-dimensional plot from before; in this case, however, we have represented the labels by both color and three-dimensional axis position. From this view, it seems reasonable that fitting a plane through this three-dimensional data would allow us to predict the expected label for any set of input parameters. Returning to the two-dimensional projection, when we fit such a plane we get the result shown in the following figure:



[figure source in Appendix \(06.00-figure-code.html#Regression-Example-Figure-3\).](#)

This plane of fit gives us what we need to predict labels for new points. Visually, we find the results shown in the following figure:



[figure source in Appendix \(06.00-figure-code.html#Regression-Example-Figure-4\)](#)

As with the classification example, this may seem rather trivial in a low number of dimensions. But the power of these methods is that they can be straightforwardly applied and evaluated in the case of data with many, many features.

For example, this is similar to the task of computing the distance to galaxies observed through a telescope—in this case, we might use the following features and labels:

- *feature 1, feature 2*, etc. $\rightarrow$ brightness of each galaxy at one of several wave lengths or colors
- *label* $\rightarrow$ distance or redshift of the galaxy

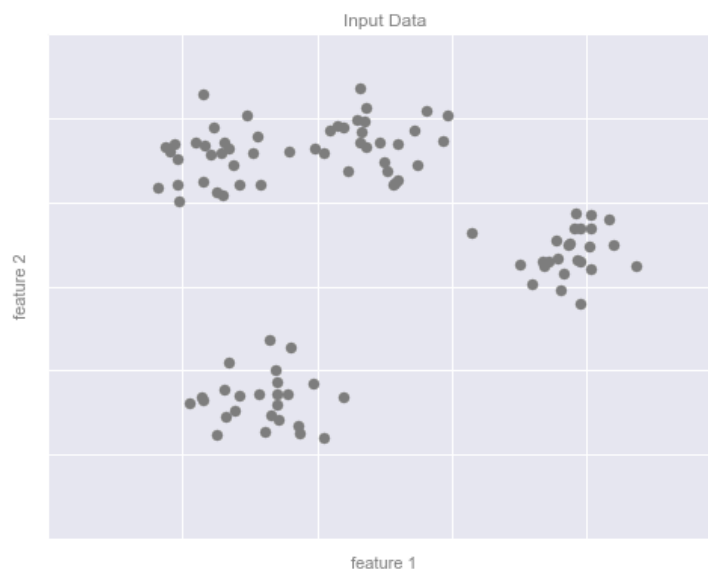
The distances for a small number of these galaxies might be determined through an independent set of (typically more expensive) observations. Distances to remaining galaxies could then be estimated using a suitable regression model, without the need to employ the more expensive observation across the entire set. In astronomy circles, this is known as the "photometric redshift" problem.

Some important regression algorithms that we will discuss are linear regression (see [In Depth: Linear Regression \(05.06-linear-regression.html\)](#)), support vector machines (see [In-Depth: Support Vector Machines \(05.07-support-vector-machines.html\)](#)), and random forest regression (see [In-Depth: Decision Trees and Random Forests \(05.08-random-forests.html\)](#)).

Clustering: Inferring labels on unlabeled data

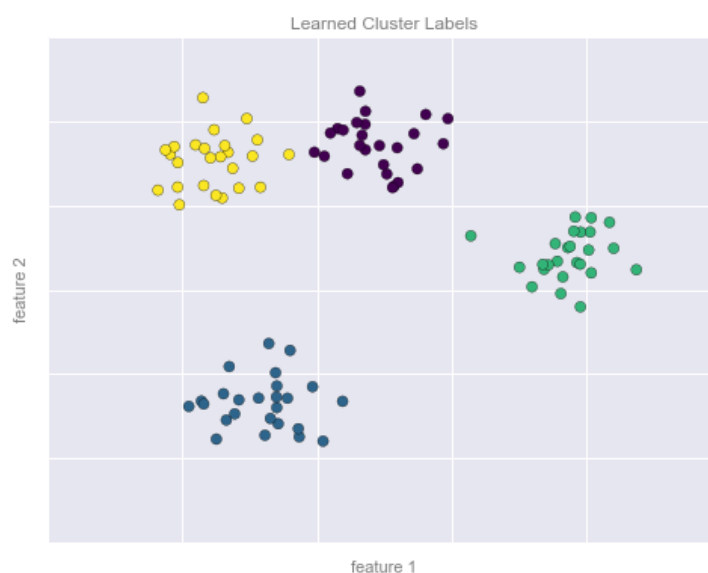
The classification and regression illustrations we just looked at are examples of supervised learning algorithms, in which we are trying to build a model that will predict labels for new data. Unsupervised learning involves models that describe data without reference to any known labels.

One common case of unsupervised learning is "clustering," in which data is automatically assigned to some number of discrete groups. For example, we might have some two-dimensional data like that shown in the following figure:



[figure source in Appendix \(06.00-figure-code.html#Clustering-Example-Figure-2\).](#)

By eye, it is clear that each of these points is part of a distinct group. Given this input, a clustering model will use the intrinsic structure of the data to determine which points are related. Using the very fast and intuitive k -means algorithm (see [In Depth: K-Means Clustering \(05.11-k-means.html\)](#)), we find the clusters shown in the following figure:



[figure source in Appendix \(06.00-figure-code.html#Clustering-Example-Figure-2\).](#)

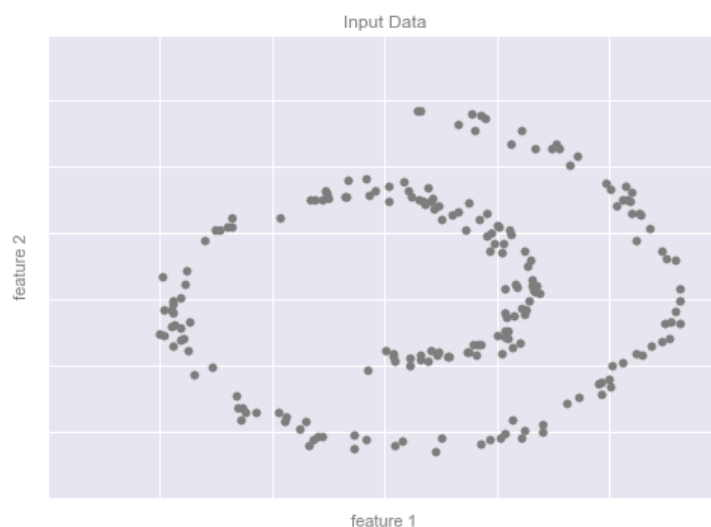
k -means fits a model consisting of k cluster centers; the optimal centers are assumed to be those that minimize the distance of each point from its assigned center. Again, this might seem like a trivial exercise in two dimensions, but as our data becomes larger and more complex, such clustering algorithms can be employed to extract useful information from the dataset.

We will discuss the k -means algorithm in more depth in [In Depth: K-Means Clustering \(05.11-k-means.html\)](#). Other important clustering algorithms include Gaussian mixture models (See [In Depth: Gaussian Mixture Models \(05.12-gaussian-mixtures.html\)](#)) and spectral clustering (See [Scikit-Learn's clustering documentation \(http://scikit-learn.org/stable/modules/clustering.html\)](#)).

Dimensionality reduction: Inferring structure of unlabeled data

Dimensionality reduction is another example of an unsupervised algorithm, in which labels or other information are inferred from the structure of the dataset itself. Dimensionality reduction is a bit more abstract than the examples we looked at before, but generally it seeks to pull out some low-dimensional representation of data that in some way preserves relevant qualities of the full dataset. Different dimensionality reduction routines measure these relevant qualities in different ways, as we will see in [In-Depth: Manifold Learning \(05.10-manifold-learning.html\)](#).

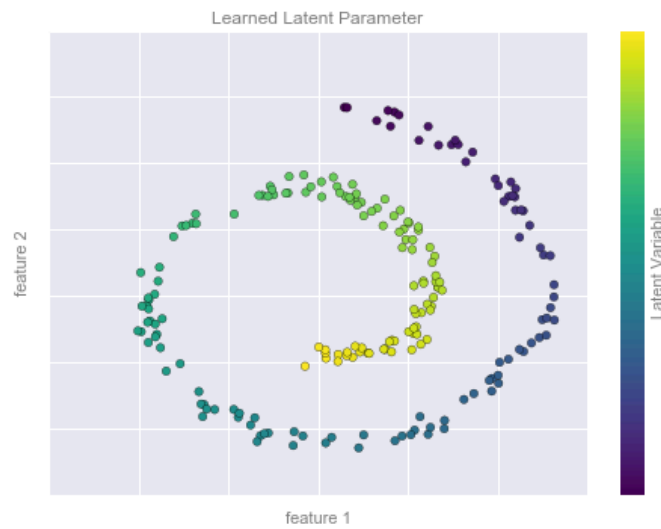
As an example of this, consider the data shown in the following figure:



[figure source in Appendix \(06.00-figure-code.html#Dimensionality-Reduction-Example-Figure-1\)](#).

Visually, it is clear that there is some structure in this data: it is drawn from a one-dimensional line that is arranged in a spiral within this two-dimensional space. In a sense, you could say that this data is "intrinsically" only one dimensional, though this one-dimensional data is embedded in higher-dimensional space. A suitable dimensionality reduction model in this case would be sensitive to this nonlinear embedded structure, and be able to pull out this lower-dimensionality representation.

The following figure shows a visualization of the results of the Isomap algorithm, a manifold learning algorithm that does exactly this:



[figure source in Appendix \(06.00-figure-code.html#Dimensionality-Reduction-Example-Figure-2\)](#).

Notice that the colors (which represent the extracted one-dimensional latent variable) change uniformly along the spiral, which indicates that the algorithm did in fact detect the structure we saw by eye. As with the previous examples, the power of dimensionality reduction algorithms becomes clearer in higher-dimensional cases. For example, we might wish to visualize important relationships within a dataset that has 100 or 1,000 features. Visualizing 1,000-dimensional data is a challenge, and one way we can make this more manageable is to use a dimensionality reduction technique to reduce the data to two or three dimensions.

Some important dimensionality reduction algorithms that we will discuss are principal component analysis (see [In Depth: Principal Component Analysis \(05.09-principal-component-analysis.html\)](#)) and various manifold learning algorithms, including Isomap and locally linear embedding (See [In-Depth: Manifold Learning \(05.10-manifold-learning.html\)](#)).

Summary

Here we have seen a few simple examples of some of the basic types of machine learning approaches. Needless to say, there are a number of important practical details that we have glossed over, but I hope this section was enough to give you a basic idea of what types of problems machine learning approaches can solve.

In short, we saw the following:

- *Supervised learning*: Models that can predict labels based on labeled training data

- *Classification*: Models that predict labels as two or more discrete categories
- *Regression*: Models that predict continuous labels
- *Unsupervised learning*: Models that identify structure in unlabeled data
 - *Clustering*: Models that detect and identify distinct groups in the data
 - *Dimensionality reduction*: Models that detect and identify lower-dimensional structure in higher-dimensional data

In the following sections we will go into much greater depth within these categories, and see some more interesting examples of where these concepts can be useful.

All of the figures in the preceding discussion are generated based on actual machine learning computations; the code behind them can be found in [Appendix: Figure Code \(06.00-figure-code.html\)](#).

< [Machine Learning \(05.00-machine-learning.html\)](#) | [Contents \(index.html\)](#) | [Introducing Scikit-Learn \(05.02-introducing-scikit-learn.html\)](#) >

 [Open in Colab](#)

<https://colab.research.google.com/github/jakevdp/PythonDataScienceHandbook/blob/master/notebooks/What-Is-Machine-Learning.ipynb>